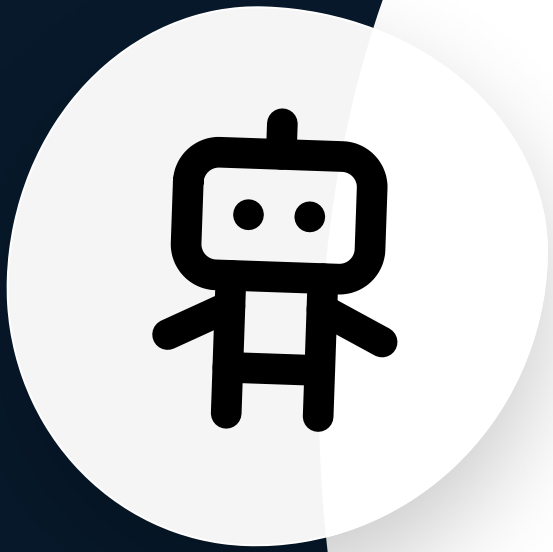
# Agentes

## M6: v0.4, Agentes



Hoy: el recorrido

# De pasos fijos a un modelo que elige




Venís de v0.3: siempre busca y responde. Hoy le das herramientas y lo dejás elegir.

Hoy cada ejemplo corre en vivo: cuaderno de la clase 5.

# Primero solo, después lo discutimos

- Control de lectura individual, 5 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre las lecturas: estado, patrones e interrupt. En la discusión, las mismas.

# Sin mirar: ¿qué hace siempre tu v0.3?



?

**¿Qué pasos?**

sigue tu v0.3, siempre en el mismo orden



?

**¿Otro camino?**

una pregunta tuya que necesite la web o una cuenta

**Respondé en una línea, sin mirar.**

¿Qué hace siempre tu `v0.3` ?

# Respuestas: pasos fijos o elige el modelo

### Siempre igual

buscar, armar el contexto, responder

### Otro camino

algo reciente (la web) o una cuenta exacta

### Hoy

que el modelo elija el camino: eso es un agente

# ¿Para qué te sirve? Que elija la herramienta justa

| Le preguntás | Usa |
| --- | --- |
| «¿Cómo se calcula el churn?» | Tus documentos |
| «¿Cuánto creció de marzo a abril?» | La calculadora |
| «¿A cuánto está hoy el dólar?» | Internet |
| Lo mismo, con la web caída | Nada: avisa, no inventa |

# Workflow: pasos fijos. Agente: elige él



## Workflow

el camino lo decidís vos · costo predecible · conviene si conocés los pasos

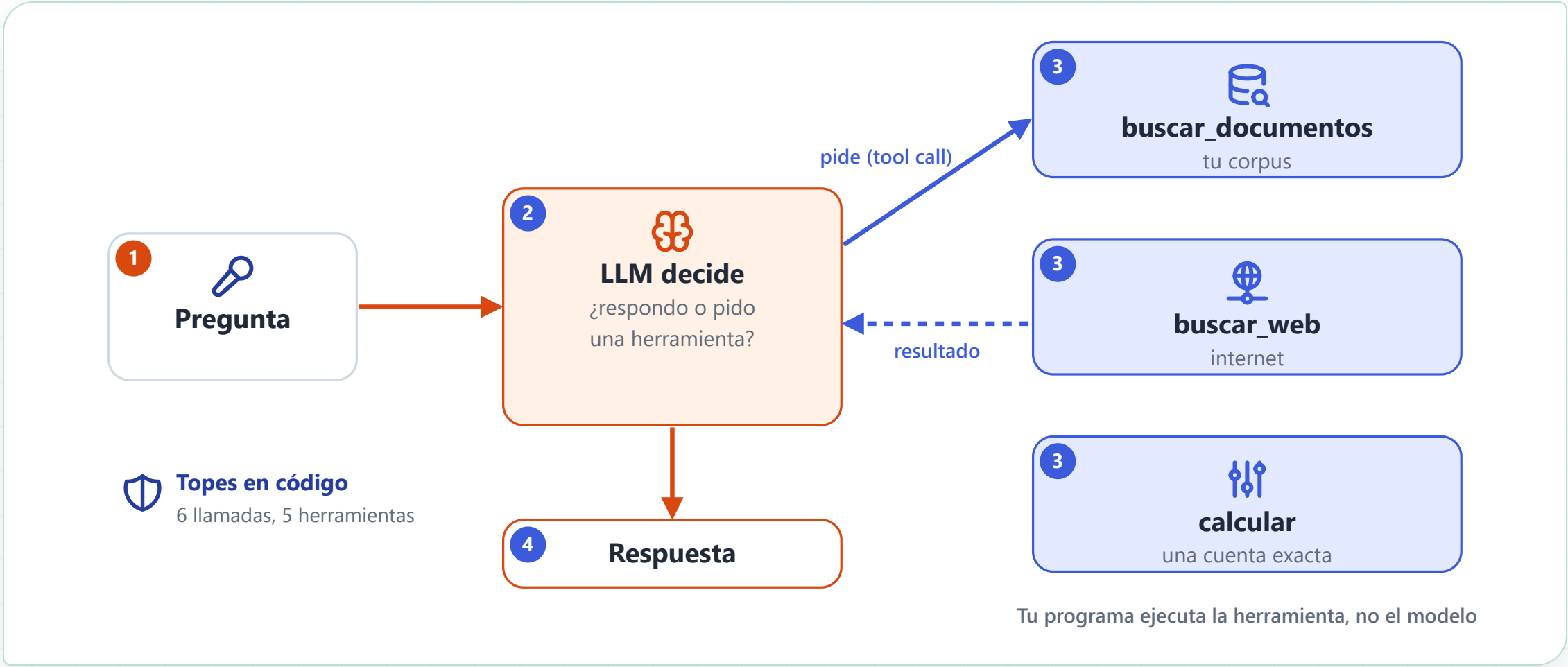


## Agente

el camino lo decide el modelo · costo variable · conviene si los pasos cambian

**Una receta, o un cocinero que mira la heladera y decide.**

# Un agente es un LLM dentro de un ciclo




Termina cuando responde o cuando llega al tope.

# Las piezas de un agente, con su nombre

| Pieza | Qué es |
|---|---|
| **Herramienta** (*tool*) | Función que el modelo pide |
| **Pedido** (*tool call*) | Nombre y datos; lo corre tu código |
| **Ciclo** (*ReAct*) | Piensa, pide, mira y repite |
| **Tope** | Cuántas llamadas, como máximo |
| **Instrucciones** | Cuándo usar cada herramienta |

# El modelo solo escribe: la herramienta hace

**Solo**
**cuentas «de cabeza»**
**no ve tus archivos**
**no sabe lo de hoy**

→

**Con herramientas, pide**
**calcular**
**buscar_documentos**
**buscar_web**

**Herramienta:** un botón con cartel. El modelo dice cuál y con qué datos; tu programa lo aprieta.

# Una herramienta tiene cuatro partes

| Parte | Para qué | En Python | ¿La ve? |
|---|---|---|---|
| Nombre | La pide así | `def calcular` | Sí |
| Descripción | Cuándo usarla | `"""Hace una cuenta…"""` | Sí |
| Parámetros | Datos y su tipo | `a: float` | Sí |
| Salida | Qué devuelve | `-> str` , `return` | Después |

El código de adentro, nunca.

# Una herramienta es una función con etiqueta

```
@tool                                          # la etiqueta: ahora es herramienta
def calcular(                                  # NOMBRE
    operacion: Literal["suma", "resta", "multiplicacion",
                       "division", "porcentaje_cambio"],  # PARÁMETROS y tipos
    a: float, b: float) -> str:                # más parámetros · SALIDA: texto
    """Hace una cuenta con dos números. … Usar para cualquier cuenta."""
    resultado = {"suma": a + b, "multiplicacion": a * b, …}[operacion]
    return f"{resultado:.12g}"                 # adentro: el modelo no lo ve
```

El texto entre `"""` es la **descripción**. `float` = número; `Literal` = opciones cerradas.

# El modelo solo ve nombre, tipos y descripción

```
{"name": "calcular",
 "description": "Hace una cuenta con dos números. porcentaje_cambio =
                 (b - a) / a * 100. Usar para cualquier cuenta.",
 "parameters": {"type": "object",
   "properties": {"operacion": {"type": "string", "enum": ["suma", "resta",
                    "multiplicacion", "division", "porcentaje_cambio"]},
                  "a": {"type": "number"}, "b": {"type": "number"}},
   "required": ["operacion", "a", "b"]}}
```

- `name`, `description`, `parameters`: las tres partes que ve.
- Nada del código de adentro: elige leyendo esto.

# El modelo pide, tu código ejecuta

```
modelo   AIMessage(content="", tool_calls=[{"name": "calcular",
                  "args": {"a": 5, "b": 3, "operacion": "multiplicacion"}}])
código   calcular(a=5, b=3, operacion="multiplicacion")  →  "15"
vuelve   ToolMessage("15"), y el modelo decide: pide otra o responde
```

- El modelo no ejecuta nada: escribe un pedido.
- Ese ciclo se llama **ReAct**.

# Así se ve el ciclo de tu v0.4, mensaje por mensaje

```
[pregunta]     …¿cuántas filas tiene la tabla que compara checkpointer y store?
               Multiplicá ese número por 3.
[modelo pide]  buscar_documentos({'consulta': 'checkpointer store tabla comparar…'})
[tu código]    buscar_documentos → '[langgraph__persistence] ## Checkpointer vs…'
[modelo pide]  calcular({'b': 3, 'operacion': 'multiplicacion', 'a': 5})
[tu código]    calcular → '15'
[respuesta]    La tabla … tiene 5 filas … el resultado es **15**.
```

- Dos herramientas, tres llamadas: dos piden, la tercera responde.
- Gemini 3.5 Flash-Lite; otra corrida: 6 × 3 = 18. En vivo: sección 3.

# La descripción vaga no cambió la elección

**Alguien dice cuándo**
**12 de 12: descripción real, o vaga con pistas**

=

**Nadie dice cuándo**
**«Busca información.», SISTEMA neutro**
**11 y 12 de 12**

gpt-5.4-nano, dos corridas: diferencia por azar. En vivo: sección 4.

# Ejecutar código: poderoso y riesgoso

### La calculadora

lo que usa tu v0.4: solo hace cuentas, con datos que tu código valida

### Ejecutar código

el modelo escribe Python y se corre: analiza datos y grafica, pero puede hacer cualquier cosa

**Ejecutar código pide un entorno aislado y que alguien apruebe.**

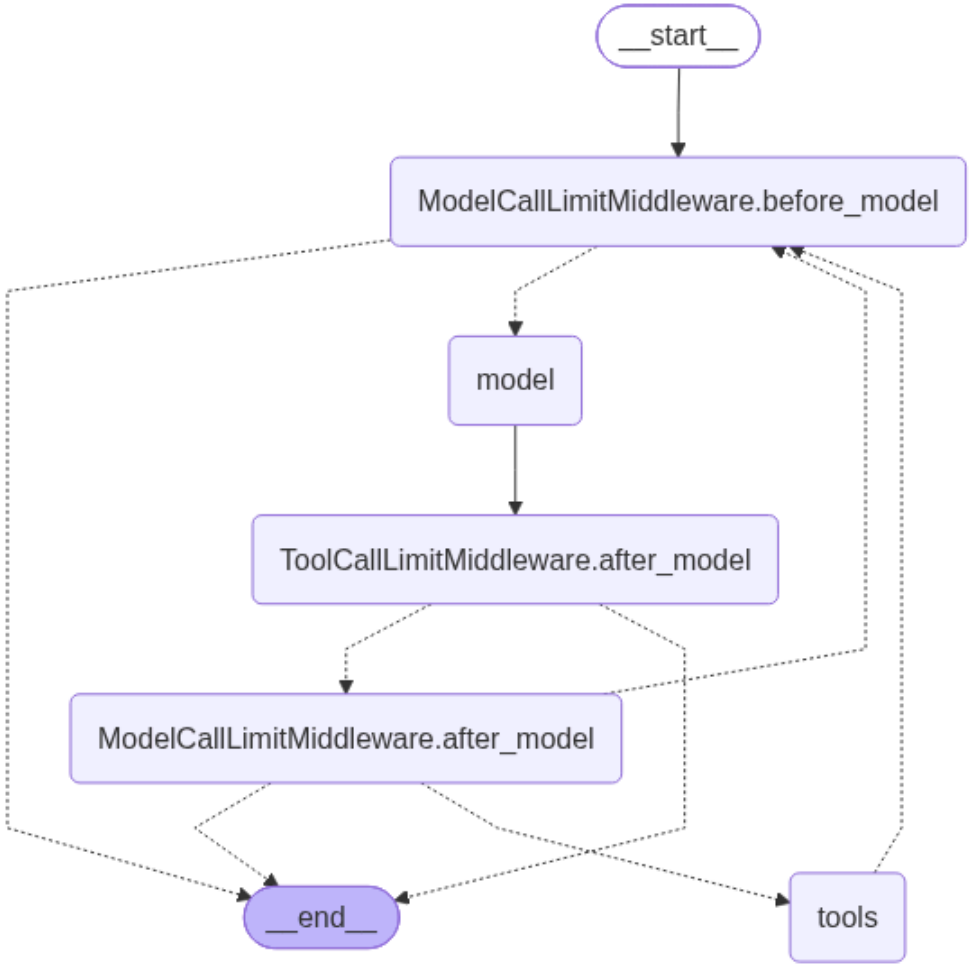
Tu agente de código lo hace: por eso pide permiso.

# Los límites van en el código, no en el prompt

```
def calcular(operacion: Literal["suma", "resta", "multiplicacion", "division",
                                "porcentaje_cambio"], a: float, b: float) -> str:
    ...  # opciones cerradas: el modelo elige una, no escribe código
agente = create_agent(modelo, tools=[buscar_documentos, buscar_web, calcular],
                      system_prompt=SISTEMA,
                      middleware=[ModelCallLimitMiddleware(run_limit=6)])
```

- Tope: hasta 6 llamadas por pregunta, aunque el prompt diga otra cosa.
- Herramientas de solo lectura: buscan y calculan, no escriben.

# En el grafo real, los topes son cajas del ciclo



# Con la web caída, uno inventó y otro avisó

**Modelo local chico**
**«La última versión es 0.1.2»**

≠

**Gemini 3.5 Flash-Lite (17/10)**
**«devolvió un error»**

- ¿Cuál liberarías?
- ¿Qué haría falta para que el chico avise?
- Pista: ¿pidió la web?

# Con la web caída, el ERROR llegó al modelo y lo dijo

```
[pregunta]     ¿Cuál es la última versión de LangGraph publicada en PyPI?
[modelo pide]  buscar_web({'consulta': 'langgraph pypi version'})
[tu código]    buscar_web → 'ERROR: la búsqueda web no respondió. No inventes; …'
[respuesta]    Lo siento, la búsqueda web devolvió un error, por lo que no puedo
               confirmar la última versión exacta de LangGraph…
```

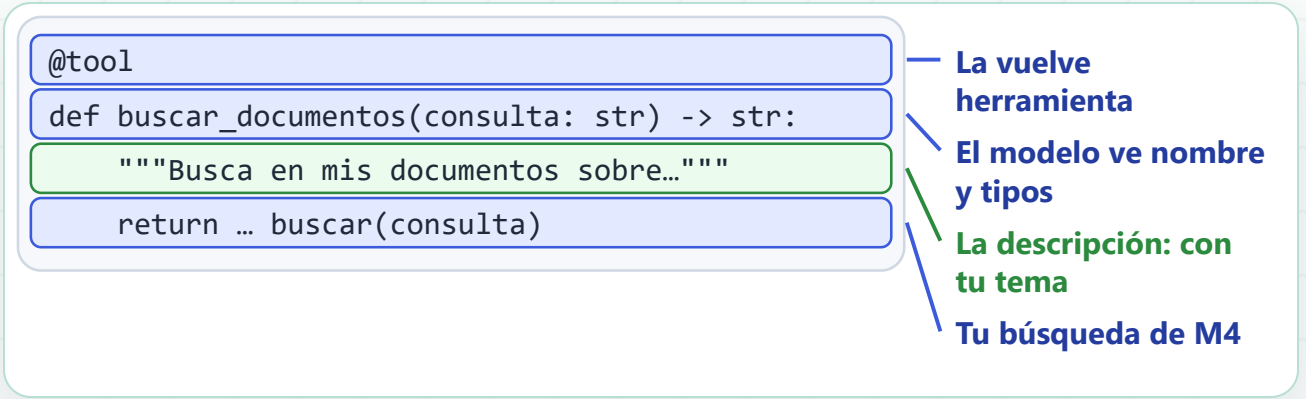
- Gemini 3.5 Flash-Lite leyó el ERROR. En vivo: sección 5.
- Si lo ignora, lo ataja el guardrail (clase 7). Si ni pide la web, no.

# Taller: tres herramientas para tu tema (55 min)

1. **Tu descripción** de `buscar_documentos`.
2. **Tu suite de selección:** `datos/mi_seleccion.jsonl`
3. **Predecí** dónde falla.

Práctica M6, arranque y «En clase», pasos 1 y 3.

**Primer control · 10–15 min:** ves qué herramienta pidió en cada pregunta.



# Mismas descripciones, otro modelo: de 9 a 12 de 12

**Modelo local chico**
**eligió bien 9 de 12**

→

**Gemini 3.5 Flash-Lite**
**12 de 12, en 3 corridas**

- Misma suite, 17/10.
- Antes de culpar a la descripción, probá otro modelo.

# ¿Por qué falló la pregunta 12?



**Esperaba**

buscar_documentos y calcular



**Hizo**

solo la primera; contó columnas (2), no filas (5): dio 6

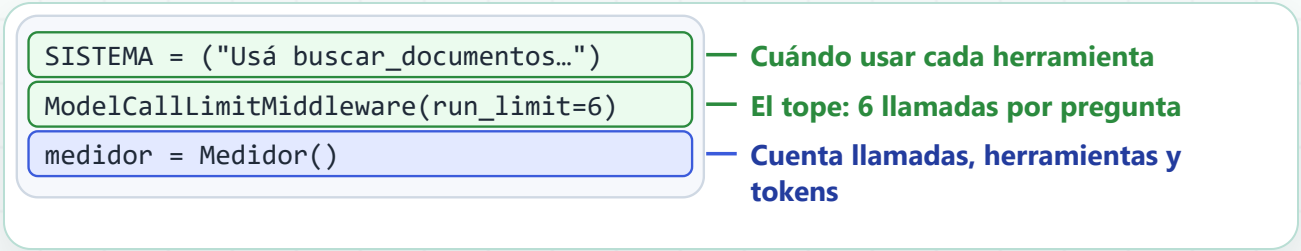


**¿Qué cambiarías?**

una sola cosa

# Taller: medir la elección y romper la web (50 min)

1. **Medí** con `web = "grabada"`
2. **Una** cosa y repetí.
3. **`web = "caida"`:** ¿avisa o inventa? Después, `"real"`




Práctica M6, «En clase», pasos 2 y 4 a 6.

**Trampa:** si ni pidió `buscar_web`, falló al elegir, no la web.

# Repaso: hoy te llevás tres ideas



**Agente**

el modelo elige el camino



**Herramienta**

tu código la ejecuta



**Topes**

en el código, no en el prompt

**El modelo elige por el nombre y la descripción de cada herramienta.**

# ¿Por qué eligió mal esta vez?

Mostrá una pregunta donde eligió mal la herramienta.

¿Fue la descripción, las instrucciones, la pregunta o el modelo?

Anotá tu **confianza (1 a 4)**. Entrega de avance 3: `evidencia/v0.3.md`, `evidencia/v0.4.md` y *push* hasta el mié 18/11, 23:59.